

STAT 547 - Data Mining

Homework 1

Rochester Institute of Technology
Ducheng Tan

February 15, 2026

Part 1: The Nine Wheels Framework

Question 1: The Wheel of Problem Formulation

A healthcare company wants to predict patient readmission within 30 days.

(a) Define this problem using the Nine Wheels terminology: $\mathcal{X}$ (input space), $\mathcal{Y}$ (output space), $\mathcal{H}$ (hypothesis space), and ℓ (loss function).

Broadly defined, a hospital readmission is when a patient who had been discharged from a hospital is admitted again to that hospital or another hospital within a specified time frame.

Let the input space $\mathcal{X}$ contain variables that the healthcare or hospital has on the patients. This can be a p dimension vector of predictor variables such as patient's demographic, age, height, historical diagnostic data, number of readmission in the past, conditions of patient at discharge date and medical measurements. The output space $\mathcal{Y}$ should be $\{0, 1\}$, 0 for no readmission and 1 for readmission within 30 days. It is clear that the output space is this way as the healthcare company wants to predict whether a patient is readmitted in 30 days or not. Our hypothesis space $\mathcal{H}$ will be defined by the set of functions that maps the input space $\mathcal{X} \rightarrow \mathcal{Y}$ under a linear combination of our predictor variables discussed in $\mathcal{X}$. In other words this is the binary logistic regression.

$$\mathcal{H} = \left\{ f_{\mathcal{H}}(x) = (1 + \exp(-(w^T x + b)))^{-1} \mid w \in \mathbb{R}^p, b \in \mathbb{R} \right\} \quad (1)$$

It follows that we use the Binary Cross-Entropy as our loss function

$$\ell(y, \hat{p}) = -(y \log \hat{p} + (1 - y) \log(1 - \hat{p})), \quad \hat{p} = f_{\mathcal{H}}(x) \in \mathcal{H} \quad (2)$$

This loss function make sense intuitively as the negative log function is larger for lower $\hat{p}$ which acts like a “penalizer” because the optimal $f^* \in \mathcal{H}$ is reached from minimizing ℓ . Also we take the negative log as our domain for $\hat{p} \in [0, 1]$ due to the sigmoid function.

$$f^* = \arg \min_{\hat{p}} \ell(y, \hat{p}) \quad (3)$$

(b) Explain why precision might be more important than accuracy here, using the business impact framework from Wheel 9.

We define precision and accuracy in the respective order.

$$\text{Precision} = \mathbb{P}(y = 1 \mid \hat{f} = 1)$$

$$\text{Accuracy} = \mathbb{P}(y = \hat{f})$$

We want to have high precision due to the fact we rather have not miss any patients for readmission. It is especially important to classify a patient as a candidate for readmission when they truly need it. Also note that it is less costly for the hospital to let go of a patient as classify the patient as a candidate for readmission even when they are truly not, since the hospital will be fined and the patient's family would most likely want to be safe than sorry. On the other hand, accuracy focus not just on the positives being classify as positive but also classifying the patient as negative when they are actually negative too. This is certainly helpful but less important, because even if we classify a true negative as positive it is less “costly” in this context.

(c) Describe what data veracity ($\mathbb{1}\{\text{data is reliable}\}$) means in healthcare and three specific challenges.

Data veracity in healthcare means that the data from the patients is reliable for the models to operate on, and is able to spit out results that is significant and accurate. Possible challenges include and are not limited to

- Missing Data: When a certain variable is not recorded from a patient such as glucose level and such.
- Noise: There is possible random error from data entry and machine measurements from the medical equipments.
- Biases: The data may not always be i.i.d as patients of a certain hospital may only represent patient of a certain region but not enough for the health company.

Question 2: The Wheel of Theoretical Guarantees

Consider the k -Nearest Neighbors (k -NN) algorithm for classification.

(a) Define the Bayes classifier $f^*(x)$ and its associated Bayes risk. Why is it the theoretical gold standard? The Bayes classifier is defined as the predictor that achieves the minimum possible risk over all possible functions in the universal space $\mathcal{Y}^{\mathcal{X}}$. This is the power set all sets of possibilities and the theoretical f_B^* is the Bayes classifier. Under the 0-1 loss $\ell_{0/1}$ the Bayes classifier is

$$f_B^* = \arg \max_{y \in \mathcal{Y}} \mathbb{P}(Y = y | X = x) \quad (4)$$

This is the theoretical gold standard because if there were any other predictor models f_o^* , then the excess risk will always be positive because f_B^* is the optimal classifier in the entire universe.

$$R(f_o^*) - R(f_B^*) > 0, \quad R(\cdot) = \mathbb{E}[\ell(y, \cdot)] = \int_{\mathcal{Y}^{\mathcal{X}}} \ell(y, \cdot) p_{XY}(x, y) dx dy \quad (5)$$

However we know in application in all cases it is almost computationally impossible or inefficient to search the entire space of $\mathcal{Y}^{\mathcal{X}}$ for this mythical f_B^* .

(b) State the universal consistency result for k -NN. What conditions on k and n are required for the k -NN risk to converge to the Bayes risk?

There are three usual conditions of k and n so that $R(f_{KNN}^*) \rightarrow R(f_B^*)$. Note that similar to the ideas of a epsilon ball in point set topology, for a new data point $x_0 \in \mathcal{X}$ we rather define a $\mathcal{V}_k(x_0)$,

$$\mathcal{V}_k(x_0) = \{x_i \in \mathcal{D}_n | d(x_0, x_i) < d_{(k)}, i \in \mathbb{N}\} \quad (6)$$

So it follows that if we have theoretically a continuous growing n (e.g $n \rightarrow \infty$) then obtain a extremely dense space that can represent our features or predictor variables, then $k \rightarrow \infty$ would allow us to have more a bigger $\mathcal{V}_k$ ball/neighborhood to catch up with the growing amount of samples. However there must be a constrain on k such that $n \gg k$ so $\frac{k}{n} \rightarrow 0$ because it would not make sense to have more neighbors than the allowable sample size n . In this context if k is too big then the metrics $d_{(i)}$'s is almost useless as it may be measuring how close x_0 is with something very far away. Thus we need n to grow faster or be much larger than k to keep the neighbors of x_0 truly "neighbors".

$$\lim_{n, k \rightarrow \infty} \mathbb{E}[R(f_{KNN}^*(n, k))] = R(f_B^*), \quad n \gg k \quad (7)$$

$$\frac{1}{k} \sum_{i \in \mathcal{V}_k(x_0)} \mathbb{1}\{y_i = 1\} \xrightarrow{P} \mathbb{P}(Y = 1 | x_0) \quad (8)$$

The above is achieved when the three conditions is well maintained.

(c) Connect this result to the bias-variance tradeoff. How does the choice of k control this tradeoff in practice, even for finite n ?

For a finite sample size n if k is small then there will be very high variance in the model as there is a lot of variations in the predictions. Say for the smallest possible k -NN we have a 1-NN, then it is plausible that if for a small change in the "location" of the x_0 we can get very different results based on the group of closest neighbor to x_0 . The bias on the other hand for this case very low as the 1-NN model is very complexed and fits the neighborhood $\mathcal{V}_1$ perfectly around x_0 excluding all other possible points in $\mathcal{X}$.

Part 2: k -Nearest Neighbors Learning

Question 3: The Distance Perspective

(a) The performance of k -NN is highly sensitive to the choice of distance metric. For a feature vector $x = (\text{age}, \text{weight}, \text{cholesterol})$, propose and justify a specific distance metric. Assume age is in years, weight in kg, and cholesterol in mg/dL.

It is clear that using the raw form of this feature vector x is not a very good idea as a distance of 10 on age is very different from the distance of 10 for the weight variable for a normal Euclidean metric. We choose to normalize our data with standardization so that we can remove the unit biases in the data. If call our standardized vector variables $\tilde{x} = (\tilde{a}, \tilde{w}, \tilde{c})$, then

$$\tilde{x} = \left(\frac{a - \frac{1}{n} \sum_{i=1}^n a_i}{\frac{1}{n-1} \sqrt{\sum_{i=1}^n (a_i - \frac{1}{n} \sum_{i=1}^n a_i)^2}}, \frac{w - \frac{1}{n} \sum_{i=1}^n w_i}{\frac{1}{n-1} \sqrt{\sum_{i=1}^n (w_i - \frac{1}{n} \sum_{i=1}^n w_i)^2}}, \frac{c - \frac{1}{n} \sum_{i=1}^n c_i}{\frac{1}{n-1} \sqrt{\sum_{i=1}^n (c_i - \frac{1}{n} \sum_{i=1}^n c_i)^2}} \right) \quad (9)$$

Since each feature will still be a real valued number after this standardization, we can say that the euclidean metric space holds from properties of the real numbers.

(b) Explain the concept of the curse of dimensionality in the context of k -NN. How does the distribution of nearest-neighbor distances change as the number of features p grows?

In the context of k -NN, the curse of dimensionality refers to the fact that as p increases, the feature space becomes exponentially sparse because volume grows at a some exponential rate of r^p . Then this will forces the k -NN algorithm to look across a much larger “volume” to find its k neighbors. Also note that even if it does find a neighbor it may actually not be a close neighbor in high dimensions under the Euclidean metric.

(c) Describe two preprocessing techniques (from Wheel 3) that are essential for effective k -NN learning and explain why.

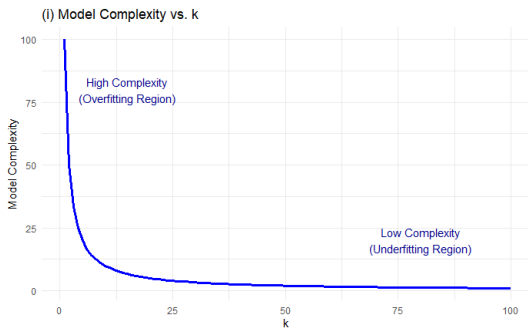
One is similar to what we have already shown above is centering which creates a centered data matrix $\bar{X}$. This is constructed by subtracting the sample mean vector $\bar{x}$ from each of the original data vectors x_i . The second one would be normalizing which is exactly what we have shown in part (3a). In this context, if we do not scale or normalize, the feature with the largest numerical range (e.g., cholesterol) would dominate the distance calculation, making the algorithm treat a minor clinical change in cholesterol as more significant than a massive demographic change in age.

Question 4: The Model Complexity Perspective

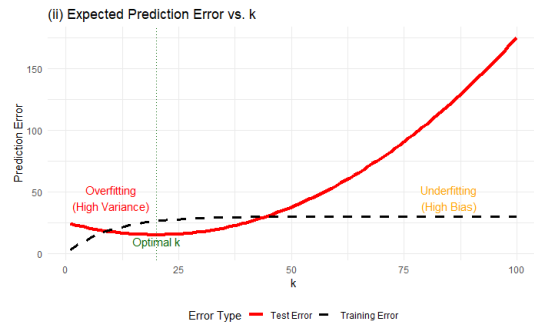
(a) In k -NN classification, how does the value of k directly control the complexity of the hypothesis space $\mathcal{H}$? Is a 1-NN model a high-capacity or low-capacity model? Justify your answer.

For small k , a high capacity model 1-NN, will try to perfectly learn every training point in the data with zero training error, however this means that the 1-NN is extremely complex for large n . On the other hand, as k grows we can see that almost all data points $x \in \mathcal{X}$ is considered (i.e $k \approx n$). For a new data point x_0 , k -NN will calculate all $d_{(k)}$ distances with x_0 . Then the question becomes what is the most popular group within the data, and f_{k-NN} will most likely point x_0 to be classified as the most frequent group in the data. Then we note that a large k in k -NN means often comes with lower complexity as it is a simpler model like almost an average function.

(b) Sketch two plots: (i) model complexity (y -axis) vs. k (x -axis), and (ii) expected prediction error (y -axis) vs. k (x -axis). Label the regions of underfitting and overfitting.



(a) kNN - Model Complexity



(b) KNN - Prediction Error

On figure (a) we plot using the usually $1/k$ relationship for Model Complexity and a increasing prediction error for large k in figure (b).

(c) Contrast k -NN (a lazy, instance-based learner) with a parametric model like logistic regression (an eager, model-based learner). List one significant advantage and one significant disadvantage of each paradigm.

This contrast with an eager learner like logistic regression, which performs the "heavy lifting" during the training phase. From (1) we have hypothesis space as a linear function (before the activation function $\sigma(z) = \frac{1}{1+e^{-z}}$) that "learns" the parameters $(\vec{w}, b)$ by minimizing a loss function like likelihood loss, which is usually computed via the log likelihood as

$$L(\vec{w}, b|X) = \prod_{1 \leq i \leq n} P(y^{(i)}|x^{(i)}; \vec{w}) = \prod_{1 \leq i \leq n} \left(\sigma(z^{(i)}) \right)^{y^{(i)}} \left(1 - \sigma(z^{(i)}) \right)^{1-y^{(i)}} \quad (10)$$

$$\ln(L(\vec{w}, b|X)) = \ell(\vec{w}, b|X) = \sum_{1 \leq i \leq n} \left[y^{(i)} \ln(\sigma(z^{(i)})) + (1 - y^{(i)}) \ln(1 - \sigma(z^{(i)})) \right] \quad (11)$$

The above is plausible when we assume that all the data has a common distribution P . From this we search for the optimal $(\vec{w}, b)$ also called the maximum likelihood estimators as

$$(\hat{\vec{w}}, \hat{b}) = \arg \max_{\vec{w}, b} \ell(\vec{w}, b|X) \quad (12)$$

Once these parameters are optimized, the eager learner can discard the training data and make predictions using only the learned weights. Advantages of logistic is clear because it is very interpretable as we understand the a unit increase corresponds to a increase in the log-odds. A remark to this is that if we assume a linear decision boundary, then if the true decision boundary is nonlinear they the model will be highly biased with large approximation error. On the other hand, the instance-based learner k -NN must store the entire dataset $\mathcal{D}_n$ to search for neighbors during inference as discussed above in (6). The k -NN has to calculate all the distances $d_{(i)}$ and use those distances on the spot to make predictions on the class membership of a new data point x_0 . The computational cost of computing these distances on something like (9) can be seen as a downside in some cases as the computational complexity per query grows with the feature dimensions. A unique advantage is that k -NN makes no parametric assumptions about the functional form of the decision boundary but creates boundaries given a sufficient amount of data.

Part 3: Contrasting Learning Paradigms

Question 5: Frequentist Gaussian Discriminant Analysis

Consider Gaussian Discriminant Analysis (GDA) for a two-class classification problem, C_1 and C_2 , with class-conditional densities $p(x|C_k) = \mathcal{N}(x|\mu_k, \Sigma)$.

(a) Derive the log-odds $\log \left(\frac{P(C_1|x)}{P(C_2|x)} \right)$ in terms of the model parameters $(\mu_1, \mu_2, \Sigma, \pi_1, \pi_2)$. Show that the decision boundary is a linear function of x .

Let the class-conditional densities $p(x|C_k)$ be defined by the multivariate gaussian distribution with the same variance covariance matrix in p dimensions. Also note that Σ is a symmetric PSD, since we will exploit properties of a symmetric PSD in the following derivations.

$$p(\mathbf{x}|\mu_k, \Sigma) = \frac{1}{(2\pi)^{p/2} |\Sigma|^{1/2}} \exp \left(-\frac{1}{2} (\mathbf{x} - \mu_k)^T \Sigma^{-1} (\mathbf{x} - \mu_k) \right) \quad (13)$$

then we can express the log-odds between the two classes C_1 and C_2 as

$$\begin{aligned} \log \left(\frac{P(C_1 | \mathbf{x})}{P(C_2 | \mathbf{x})} \right) &= \log \left(\frac{p(C_1)}{p(C_2)} \cdot \frac{p(\mathbf{x} | C_1)}{p(\mathbf{x} | C_2)} \right) \\ &= \log \left(\frac{\pi_1}{\pi_2} \right) + \log \left(\frac{p(\mathbf{x} | C_1)}{p(\mathbf{x} | C_2)} \right) \end{aligned} \quad (14)$$

Replacing π_1, π_2 to the prior of the being in class and using properties of logarithms we obtain

$$\begin{aligned} \log \left(\frac{P(C_1 | \mathbf{x})}{P(C_2 | \mathbf{x})} \right) &= \log \left(\frac{\pi_1}{\pi_2} \right) + \log \left(\frac{1}{(2\pi)^{p/2} |\Sigma|^{1/2}} \exp \left(-\frac{1}{2} (\mathbf{x} - \mu_1)^T \Sigma^{-1} (\mathbf{x} - \mu_1) \right) \right) \\ &\quad - \log \left(\frac{1}{(2\pi)^{p/2} |\Sigma|^{1/2}} \exp \left(-\frac{1}{2} (\mathbf{x} - \mu_2)^T \Sigma^{-1} (\mathbf{x} - \mu_2) \right) \right) \\ &= \log \left(\frac{\pi_1}{\pi_2} \right) + \left(-\frac{1}{2} (\mathbf{x} - \mu_1)^T \Sigma^{-1} (\mathbf{x} - \mu_1) \right) - \left(-\frac{1}{2} (\mathbf{x} - \mu_2)^T \Sigma^{-1} (\mathbf{x} - \mu_2) \right) \end{aligned} \quad (15)$$

Taking the transpose of a scalar $\mathbf{x}^T \Sigma \mu_k$ yields $\mu_k^T \Sigma \mathbf{x}$

$$\begin{aligned} &= \log \left(\frac{\pi_1}{\pi_2} \right) - \frac{1}{2} (\mathbf{x}^T \Sigma^{-1} \mathbf{x} - 2\mathbf{x}^T \Sigma^{-1} \mu_1 + \mu_1^T \Sigma^{-1} \mu_1) + \frac{1}{2} (\mathbf{x}^T \Sigma^{-1} \mathbf{x} - 2\mathbf{x}^T \Sigma^{-1} \mu_2 + \mu_2^T \Sigma^{-1} \mu_2) \\ &= \mathbf{x}^T \Sigma^{-1} (\mu_1 - \mu_2) - \frac{1}{2} \mu_1^T \Sigma^{-1} \mu_1 + \frac{1}{2} \mu_2^T \Sigma^{-1} \mu_2 + \log \left(\frac{\pi_1}{\pi_2} \right) \end{aligned} \quad (16)$$

The decision bound is a linear function of $\mathbf{x}$ can be seen as

$$\log \left(\frac{P(C_1 | \mathbf{x})}{P(C_2 | \mathbf{x})} \right) = \gamma_0 + \gamma^T \mathbf{x}, \quad \gamma = \Sigma^{-1} (\mu_1 - \mu_2), \quad \gamma_0 = -\frac{1}{2} \mu_1^T \Sigma^{-1} \mu_1 + \frac{1}{2} \mu_2^T \Sigma^{-1} \mu_2 + \log \left(\frac{\pi_1}{\pi_2} \right) \quad (17)$$

(b) Write down the closed-form expressions for the Maximum Likelihood Estimates (MLE) of the parameters μ_1, μ_2 , and the shared covariance matrix Σ . Given a dataset $\{(\mathbf{x}_i, y_i)\}_{i=1}^N$ where $y_i \in \{C_1, C_2\}$, let N_1 and N_2 denote the number of samples in classes C_1 and C_2 respectively, with $N_1 + N_2 = N$. Then likelihood function is

$$L(\mu_1, \mu_2, \Sigma, \pi_1, \pi_2) = \prod_{i=1}^N P(y_i) p(\mathbf{x}_i | y_i) \quad (18)$$

Taking the logarithm and breaking up the sum into a sum over its respective class yields

$$\begin{aligned} \ell &= \log L = \sum_{i=1}^N [\log P(y_i) + \log p(\mathbf{x}_i | y_i)] \\ &= \sum_{i|y_i=C_1} [\log \pi_1 + \log p(\mathbf{x}_i | C_1)] + \sum_{i|y_i=C_2} [\log \pi_2 + \log p(\mathbf{x}_i | C_2)] \end{aligned} \quad (19)$$

with the Gaussian density

$$\begin{aligned} \ell &= \sum_{i|y_i=C_1} \left[\log \pi_1 - \frac{p}{2} \log(2\pi) - \frac{1}{2} \log |\Sigma| - \frac{1}{2} (\mathbf{x}_i - \mu_1)^T \Sigma^{-1} (\mathbf{x}_i - \mu_1) \right] \\ &+ \sum_{i|y_i=C_2} \left[\log \pi_2 - \frac{p}{2} \log(2\pi) - \frac{1}{2} \log |\Sigma| - \frac{1}{2} (\mathbf{x}_i - \mu_2)^T \Sigma^{-1} (\mathbf{x}_i - \mu_2) \right] \end{aligned} \quad (20)$$

Here we will assume that the likelihood function is convex, the proof involves the hessian $\nabla^2 \ell(\mu_1, \mu_2)$ which may drift us too far away from this derivation. Anyways, taking the partial of ℓ with respect to μ_k for $k = 1, 2$. The steps here are skipped but notice that for partial derivative respective to μ_1 we can turn the second term of the sum in (20) and the constants within the sum over the set $i|y_i = C_1$ to 0. This simplification turns the problem into $\frac{\partial (\mathbf{x}_i - \mu_1)^T \Sigma^{-1} (\mathbf{x}_i - \mu_1)}{\partial \mu_1}$, we continue from here by expanding the quadratic form and conveniently for us this simplification can be applied to the sum of the C_2 class due to symmetry. Setting both partial to zero we obtain

$$\frac{\partial \ell}{\partial \mu_1} = \sum_{i|y_i=C_1} \Sigma^{-1} (\mathbf{x}_i - \mu_1) = \mathbf{0} \quad (21)$$

$$\frac{\partial \ell}{\partial \mu_2} = \sum_{i|y_i=C_2} \Sigma^{-1} (\mathbf{x}_i - \mu_2) = \mathbf{0} \quad (22)$$

$$\Sigma^{-1} \sum_{i|y_i=C_1} \mathbf{x}_i = \Sigma^{-1} N_1 \mu_1 \xrightarrow{\text{pre-multiply } \Sigma} \hat{\mu}_1 = \frac{1}{N_1} \sum_{i|y_i=C_1} \mathbf{x}_i, \quad \hat{\mu}_2 = \frac{1}{N_2} \sum_{i|y_i=C_2} \mathbf{x}_i \quad (23)$$

The log-likelihood terms involving Σ in the first term in which we rewrite for better visualization

$$\ell_\Sigma = -\frac{N}{2} \log |\Sigma| - \frac{1}{2} \sum_{k=1}^2 \sum_{i|y_i=C_k} (\mathbf{x}_i - \mu_k)^T \Sigma^{-1} (\mathbf{x}_i - \mu_k) \quad (24)$$

Using the matrix derivative identities $\frac{\partial}{\partial \Sigma^{-1}} \log |\Sigma| = -\Sigma$ and $\frac{\partial}{\partial \Sigma^{-1}} \text{tr}(\mathbf{A} \Sigma^{-1}) = \mathbf{A}^T$

$$\frac{\partial \ell}{\partial \Sigma^{-1}} = -\frac{N}{2} \Sigma + \frac{1}{2} \sum_{k=1}^2 \sum_{i|y_i=C_k} (\mathbf{x}_i - \boldsymbol{\mu}_k)(\mathbf{x}_i - \boldsymbol{\mu}_k)^T = \mathbf{0} \quad (25)$$

$$\hat{\Sigma} = \frac{1}{N} \sum_{k=1}^2 \sum_{i|y_i=C_k} (\mathbf{x}_i - \hat{\boldsymbol{\mu}}_k)(\mathbf{x}_i - \hat{\boldsymbol{\mu}}_k)^T \quad (26)$$

We acknowledge there are multiple approaches to maximize the MLE for π_1, π_2 we can solve this with the Lagrange multiplier method with the constraint $\pi_1 + \pi_2 = 1$. We will just simply follow the same motion as before by maximizing the likelihood $\ell(\pi_1, \pi_2) = N_1 \log \pi_1 + N_2 \log \pi_2$, and let $\pi_1 = 1 - \pi_2$

$$\ell(\pi_2) = N_1 \log(1 - \pi_2) + N_2 \log \pi_2 \quad (27)$$

$$\frac{\partial \ell(\pi_2)}{\partial \pi_2} = \frac{-N_1}{1 - \pi_2} + \frac{N_2}{\pi_2} = 0 \quad (28)$$

$$N_1 \pi_2 = N_2(1 - \pi_2) \Rightarrow (N_1 + N_2) \pi_2 = N_2 \quad (29)$$

Again by symmetry we could have found the MLE for π_1 with $\pi_2 = 1 - \pi_1$ first, so get the other one for free.

$$\hat{\pi}_1 = \frac{N_1}{N}, \quad \hat{\pi}_2 = \frac{N_2}{N} \quad (30)$$

(c) Interpret the MLE for Σ . How does it pool information from both classes, and what is a key statistical advantage of this pooling compared to having separate covariances?

It is easy to notice that the MLE for Σ in (26) has the form

$$\hat{\Sigma} = \frac{N_1}{N} \hat{\Sigma}_1 + \frac{N_2}{N} \hat{\Sigma}_2 \quad (31)$$

where $\hat{\Sigma}_k = \frac{1}{N_k} \sum_{i|y_i=C_k} (\mathbf{x}_i - \hat{\boldsymbol{\mu}}_k)(\mathbf{x}_i - \hat{\boldsymbol{\mu}}_k)^T$ is the sample covariance matrix for class k . This pooled estimator combines information from both classes, weighted by their respective sample sizes. A key statistical advantage of this pooling compared to estimating separate covariances $\hat{\Sigma}_1$ and $\hat{\Sigma}_2$ is improved stability and reduced variance of the estimator. When estimating a $p \times p$ covariance matrix, we must estimate $\frac{p(p+1)}{2}$ (This is the triangle with the diagonal from a $p \times p$ grid) unique parameters. For small sample sizes within individual classes (especially when $N_k \approx p$), separate covariance estimates can be poorly conditioned or even singular. By pooling, we effectively use all $N = N_1 + N_2$ samples to estimate a single shared Σ , which substantially increases the effective sample size per parameter and leads to a more reliable, well-conditioned estimator.

Question 6: Computational Complexity and Scalability

(a) Analyze the computational complexity of the standard k -NN algorithm for making a single prediction. Break it down into the complexity of (i) calculating distances and (ii) finding the k smallest distances. For a single prediction on a new data point $x_0 \in \mathbb{R}^p$, given a training dataset $\mathcal{D}_n = \{(x_i, Y_i)\}_{i=1}^n$, the k -NN algorithm requires two main computational steps:

(i) **Calculating distances:** To compute the distance $d(x_0, x_i)$ for each training point x_i , we typically use the Euclidean metric (a specific case of the Minkowski distance)

$$d(x_0, x_i) = \sqrt{\sum_{j=1}^p (x_{0j} - x_{ij})^2} \quad (32)$$

For each distance calculation, we perform p subtractions, p squarings, $p - 1$ additions, and one square root operation. This gives $O(3p - 1) \sim O(p)$ operations per distance. Since we must compute distances to all n training points, the total complexity for this step is $O(np)$.

(ii) **Finding the k smallest distances:** After computing all n distances, we need to identify the k nearest neighbors. Using a naive approach of sorting all distances requires $O(n \log n)$ comparisons. Alternatively, using a min-heap or partial sorting (such as quickselect) to find the k smallest elements requires $O(n \log k)$ operations. For practical cases where $k \ll n$, this step still contributes $O(n \log k)$ complexity. Therefore, the total computational complexity for a single prediction is

$$O(np) + O(n \log k) = O(np + n \log k) \quad (33)$$

In most practical scenarios where p is moderately large, the distance calculation dominates, giving an effective complexity of $O(np)$ per prediction.

(b) The standard k -NN algorithm has a time complexity of $O(nd)$ for a single prediction, where n is the training set size and d is the number of dimensions. Explain why this does not scale well for real-time applications with large n and d .

The $O(nd)$ complexity presents severe scalability challenges for real-time applications due to several factors: Linear growth with dataset size: As the training set grows, every additional sample directly increases prediction time. For modern datasets with n in the millions or billions, computing distances to every point becomes expensive. When d is large (e.g., $d = 1000$ for high-dimensional feature vectors), the distance computation itself becomes expensive. Moreover, the curse of dimensionality causes distances to become less meaningful as d grows in high dimensions, all points tend to be approximately equidistant from each other, thus worsens the quality of nearest neighbor identification.

(c) Research and describe one algorithmic strategy (e.g., KD-Trees, Ball Trees, approximate nearest neighbors) used to accelerate the nearest neighbor search. Explain its core idea and under what conditions it is most effective.

KD-Trees (k-dimensional trees) are a widely-used data structure for accelerating nearest neighbor search by exploiting spatial partitioning of the feature space. The core idea is that a KD-Tree recursively partitions $\mathbb{R}^p$ into axis-aligned hyperrectangular regions by splitting the data along alternating coordinate axes. At each level of the tree, we select a dimension $j \in \{1, \dots, p\}$ (typically cycling through dimensions or choosing the one with highest variance), choose a split value s (often the median of the data along dimension j), and partition points into left subtree ($x_{ij} \leq s$) and right subtree ($x_{ij} > s$). This creates a binary tree of depth $O(\log n)$, with each leaf containing a small number of points. To find the k nearest neighbors of x_0 , we traverse the tree to find the leaf region containing x_0 at a cost of $O(\log n)$, maintain a priority queue of the current k best candidates, and backtrack through the tree while pruning entire subtrees whose bounding boxes cannot contain closer points than the current k -th nearest neighbor.

The key insight is that if the distance from x_0 to the k -th nearest neighbor is less than the distance from x_0 to the hyperplane defining a subtree's boundary, we can safely prune that entire subtree without examining any of its points. In the best case, a KD-Tree reduces query time from $O(np)$ to $O(k \log n)$, a dramatic improvement for large n .

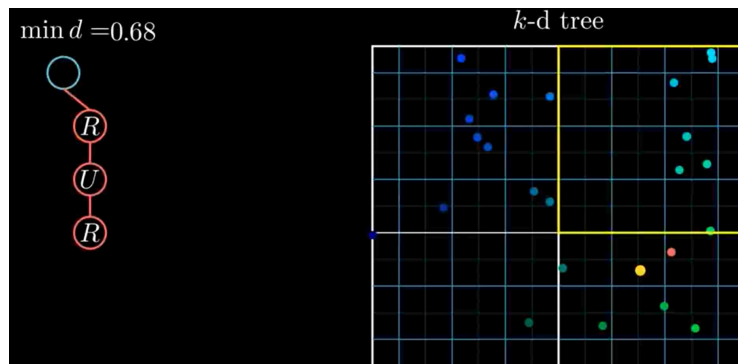


Figure 1: KD-Trees - Baqablh

Part 4: Computational Demonstration

Question 7: Simulation Insights

(a) Implement the following simulation to illuminate the bias-variance tradeoff in k-NN, and then answer the following question: What does this visualization teach about the bias-variance tradeoff in k-NN? How does this connect to Question 4?

Recall that the k-NN regression assumes $\mathcal{D}_n = \{(x_i, Y_i)\}_{i=1}^n$ with $x_i = (x_{i1}, \dots, x_{ip})^T \in \mathcal{X}^p, Y_i \in \mathbb{R}$. We generated artificial the data with $n = 100$ samples from a uniform distribution on the continuous interval $[0, 2\pi]$, so $x \sim U[0, 2\pi]$ and with $Y_i = \sin(x_i) + \varepsilon_i$ where $\varepsilon_i \sim N(\mu = 0, \sigma = 0.3)$. Then we can plot the true generator of the data as $\sin(x)$ to compare against the kNN regression $\hat{Y}_{kNN}$.

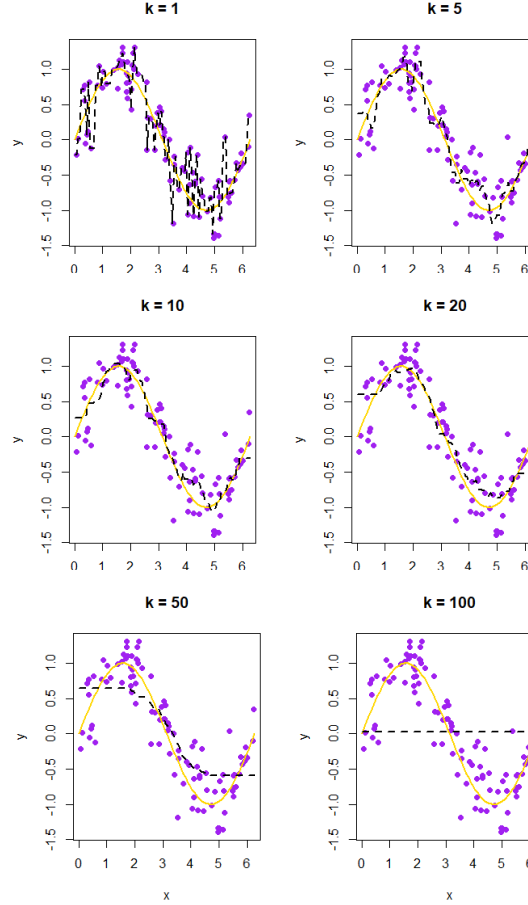


Figure 2: Visual Performance of k-NN Regressed on Y_i

After running the kNN regression we can see that for smaller k like $k = 1, 5$, the kNN regression line in black attempts to trace in every possible point given in $\mathcal{D}_n$, in other words “memorizing” the data. Then for $k = 10, 20$, the k-NN regression line performs extremely close to the true generator of $\mathcal{D}_n$. Lastly, for $k = 50, 100$, it is apparent that the regression line has been smoother out by the choice of such large k . The reason being that in the computation of $\hat{f}_{kNN}(x_0)$ for a new x_0 , it is essentially taking the average of all points in $\mathcal{V}_k(x_0)$, and specifically for $k = 100$, it is clear the that average of $\sin(x)$ is 0 on $[0, 2\pi]$. (i.e. $\frac{1}{2\pi} \int_{C[0, 2\pi]} \sin(x) dx = 0$)

$$\hat{Y}_{kNN} = \hat{f}_{kNN}(x_0) = \frac{1}{k} \sum_{x_i \in \mathcal{V}_k(x_0)} Y_i = \frac{1}{k} \sum_{i=1}^n Y_i \mathbb{1}(x_i \in \mathcal{V}_k(x_0)) \quad (34)$$

To answer the question, this visualization of kNN regression on the artificially generate data points to a clear demonstration of the bias-variance tradeoff. This transition from $k = 1$ to $k = 100$ represents a shift from high variance and low bias to low variance high bias. This is a direct application of the discussion in Question (4).

(b) Implement the following simulation to contrast k-NN and GDA on a simple dataset, and then answer the following question: Compare and contrast the decision boundaries. What does this tell you about the inherent assumptions of k-NN vs. GDA? For the classification task, we consider a dataset $\mathcal{D}_n = \{(x_i, Y_i)\}_{i=1}^{2n}$ where $x_i = (x_{i1}, x_{i2})^T \in \mathbb{R}^2$ and $Y_i \in \{1, 2\}$ denotes the class label. Next, we generated $n = 50$ samples from each of two classes using multivariate normal distributions

$$C_1 : x_i \sim \mathcal{N}_2 \left(\mu_1 = \begin{pmatrix} 0 \\ 0 \end{pmatrix}, \Sigma_1 = \begin{pmatrix} 1 & 0.5 \\ 0.5 & 1 \end{pmatrix} \right), \quad i = 1, \dots, 50$$

$$C_2 : x_i \sim \mathcal{N}_2 \left(\mu_2 = \begin{pmatrix} 1.5 \\ 1.5 \end{pmatrix}, \Sigma_2 = \begin{pmatrix} 1 & 0.5 \\ 0.5 & 1 \end{pmatrix} \right), \quad i = 51, \dots, 100$$

Notice that both classes share the same covariance matrix $\Sigma_1 = \Sigma_2 = \Sigma$, which satisfies the key assumption of Linear Discriminant Analysis (LDA). Under this assumption, the Bayes optimal decision boundary has the linear form.

$$\delta_c(x) = x^T \Sigma^{-1} \mu_c - \frac{1}{2} \mu_c^T \Sigma^{-1} \mu_c + \log \pi_c \quad (35)$$

We compare two classification methods from plotting their respective boundaries on the data $\mathcal{D}_n$. For $k = 5$, the predicted class from 5NN is

$$\hat{Y}_{5NN}(x) = \text{mode}\{Y_i : x_i \in \mathcal{V}_5(x)\}$$

A parametric method that assumes Gaussian class-conditional densities with equal covariance matrices. The decision boundary is linear and given by the line

$$\mathbb{L} = \{x \in (X_1 \times X_2) : \delta_1(x) = \delta_2(x)\} \quad (36)$$

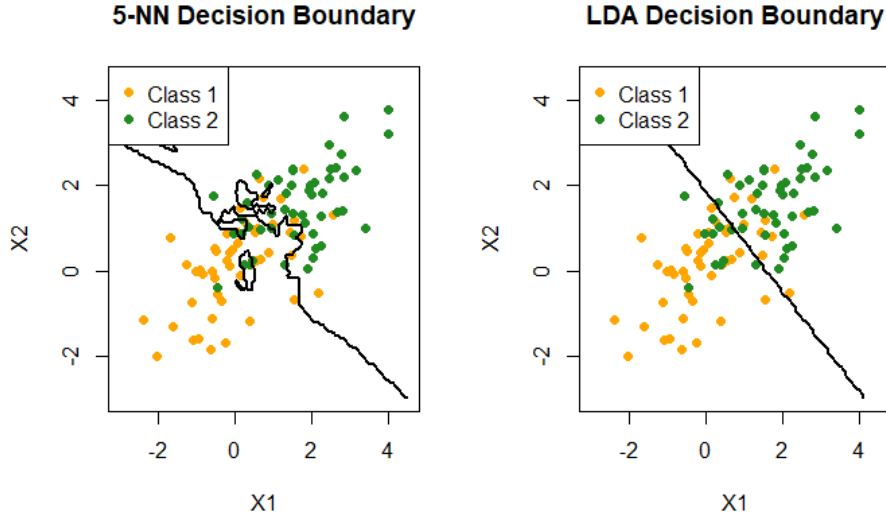


Figure 3: 5NN and LDA on $\mathcal{D}_n$. The 5NN has a decision boundary that traces out region to separates most of the two classes correctly in area where both classes overlaps. The LDA decision boundary is a clear linear line that splits the two classes into two groups without complex decision lines like 5-NN.

A remark is that LDA is born for this type of data set as the two variances matrices for the two classes are exactly the same. LDA should generalize better on new test data from the same distribution, and note that for large n LDA would most likely out perform 5-NN. An interesting observation is that if we take k to be large, then the decision boundary would approach $\mathbb{L}$ in LDA !

Question 8: Video Presentation

(a) Explain one key insight from Questions 1-6 that surprised you (e.g., the universal consistency of k-NN, the curse of dimensionality, or the ethical implications). Connect the theoretical guarantee of k-NN (from Q2) to the practical challenge of the curse of dimensionality (from Q3). Discuss what “AI-aware” statistics means to you personally. Propose one way this course could better prepare students for the AI future.

Part 5: Idiosyncratic Reflection (Mandatory)**Question 9: Self-Assessment**

(a) In your own words, explain the concepts from the first few weeks of STAT 547 that you feel you have truly mastered. What is the most challenging concept or practical skill (e.g., R coding, theory, interpretation) that you are still working to solidify?

(b) After completing this homework assignment, what grade (e.g., A, B, C, etc.) do you estimate you will earn? Justify your estimate by referencing which exercises you found straightforward and which you found most challenging.